

Что такое база данных?

База данных (БД) — это организованная совокупность данных, которая хранится и управляется с помощью системы управления базами данных (СУБД). Эти данные могут быть структурированы в виде таблиц, записей, полей и связей между ними, что позволяет эффективно сохранять, извлекать, обновлять и управлять информацией.

Основные компоненты базы данных:

- **Таблицы:** Основная структура для хранения данных, где каждая таблица состоит из строк (записей) и столбцов (полей).
- **Записи:** Строки в таблице, каждая из которых представляет собой отдельный объект данных (например, один пользователь или одну транзакцию).
- **Поля:** Столбцы в таблице, представляющие отдельные характеристики данных (например, имя пользователя, дата транзакции).
- **Ключи:** Поля или комбинации полей, используемые для уникальной идентификации записей в таблице и установления связей между разными таблицами (например, первичный ключ, внешний ключ).
- **Связи:** Определенные правила, которые устанавливают отношения между таблицами, позволяя организовывать данные более эффективно и логично (например, отношения "один ко многим", "многие ко многим").

Что такое СУБД?

Система управления базами данных (СУБД) — это программное обеспечение, которое используется для создания, управления, и взаимодействия с базами данных. Она предоставляет интерфейс между пользователем (или приложением) и самой базой данных, позволяя пользователю выполнять различные операции с данными, такие как их создание, чтение, обновление и удаление (CRUD).

Основные функции и задачи СУБД:

- **Управление данными:** СУБД предоставляет инструменты для определения структуры базы данных (например, создание таблиц, определение связей), а также для управления самими данными (вставка, обновление, удаление, поиск).
- **Обеспечение целостности данных:** СУБД контролирует, чтобы данные оставались корректными и согласованными. Например, она предотвращает создание дублирующих записей или нарушение связей между таблицами.
- **Обеспечение безопасности данных:** СУБД предоставляет механизмы контроля доступа, которые позволяют ограничить права пользователей на чтение, изменение или удаление данных. Это важно для защиты данных от несанкционированного доступа или изменений.
- **Поддержка многопользовательского доступа:** СУБД обеспечивает возможность одновременной работы с базой данных для нескольких пользователей или приложений. При этом она предотвращает конфликты, возникающие из-за одновременных изменений одних и тех же данных.
- **Резервное копирование и восстановление данных:** СУБД обычно включает функции для автоматического создания резервных копий базы данных и восстановления данных в случае сбоя системы.
- **Оптимизация запросов:** СУБД анализирует и оптимизирует запросы к базе данных, чтобы повысить производительность, особенно при работе с большими объемами данных.

Зачем нужна СУБД?

- Эффективное управление данными: СУБД упрощает процесс управления большими объемами данных, автоматизируя многие задачи, такие как индексация, сортировка и фильтрация данных.
- Централизованное хранение данных: Данные хранятся централизованно, что обеспечивает легкий доступ к ним, их управляемость и безопасность.
- Совместная работа с данными: Многопользовательский доступ позволяет нескольким пользователям одновременно работать с базой данных без риска потери данных или возникновения конфликтов.
- Повышение безопасности: СУБД позволяет ограничивать доступ к данным, обеспечивая их защиту и конфиденциальность.
- Поддержка сложных запросов и отчетности: СУБД позволяет выполнять сложные запросы к базе данных и генерировать отчеты на основе имеющихся данных.

Примеры СУБД

- MySQL: Открытая и бесплатная СУБД, популярная для веб-приложений.
- PostgreSQL: Мощная и расширяемая СУБД с поддержкой сложных запросов.
- Oracle Database: Коммерческая СУБД, известная своей высокой производительностью и масштабируемостью.
- Microsoft SQL Server: СУБД от Microsoft, широко используемая в корпоративных приложениях.
- MongoDB: Нереляционная (NoSQL) СУБД, предназначенная для работы с неструктурированными данными.

Реляционная база данных (РБД)

Реляционная база данных (РБД) организует данные в виде таблиц (или отношений), где строки представляют записи, а столбцы – атрибуты этих записей. Каждая таблица в РБД имеет уникальный ключ (обычно первичный ключ), который однозначно идентифицирует каждую запись. Таблицы могут быть связаны друг с другом через внешние ключи, что позволяет эффективно управлять связанными данными.

Основные характеристики:

- Таблицы: Данные организованы в виде таблиц, где каждая таблица состоит из строк и столбцов.
- SQL (Structured Query Language): Для управления данными в РБД используется язык SQL, который позволяет выполнять запросы, вставку, обновление и удаление данных.
- Нормализация данных: Процесс нормализации используется для минимизации избыточности данных и обеспечения целостности данных.
- Целостность данных: РБД обеспечивает целостность данных через ограничения, такие как первичные и внешние ключи, уникальность и т.д.

Примеры реляционных СУБД:

- MySQL
- PostgreSQL
- Oracle Database
- Microsoft SQL Server

Примеры использования:

- Финансовые системы
- Системы управления запасами

- Корпоративные приложения (CRM, ERP)

Нереляционная база данных (NoSQL)

Нереляционная база данных (NoSQL) – это тип базы данных, которая не использует традиционную табличную структуру для организации данных. Вместо этого она может использовать различные модели данных, такие как документы, ключ-значение, графы или столбцы. Нереляционные базы данных обычно лучше подходят для работы с большими объемами неструктурированных или слабо структурированных данных.

Основные характеристики:

- Модели данных: Нереляционные базы данных могут использовать разные модели данных:
 - Документные: данные хранятся в виде документов, например, JSON или BSON (например, MongoDB).
 - Ключ-значение: данные хранятся в виде пар ключ-значение (например, Redis).
 - Графовые: данные представлены в виде вершин и ребер, что позволяет моделировать сложные отношения (например, Neo4j).
 - Столбцовые: данные хранятся в виде столбцов, а не строк (например, Apache Cassandra).
- Гибкость структуры данных: Нереляционные базы данных часто менее строго типизированы, что позволяет хранить данные с разными структурами в одной коллекции или таблице.
- Масштабируемость: Они легче масштабируются горизонтально, что делает их подходящими для обработки больших объемов данных и высоких нагрузок.
- Отсутствие SQL: Взаимодействие с данными происходит с использованием API или специализированных языков запросов, а не SQL.

Примеры нереляционных СУБД:

- MongoDB (документная)
- Redis (ключ-значение)
- Neo4j (графовая)
- Apache Cassandra (столбцовая)

Примеры использования:

- Хранение больших объемов неструктурированных данных (например, данные социальных сетей).
- Реализация кеширования и управления сессиями (например, Redis).
- Работа с графами и связями (например, социальные сети или рекомендации).
- Приложения с высокими требованиями к масштабируемости и производительности.

Сравнение реляционных и нереляционных баз данных

Характеристика	Реляционные БД	Нереляционные БД
Структура данных	Таблицы, строки и столбцы	Документы, ключ-значение, графы, столбцы
Язык запросов	SQL	API или специализированные языки запросов
Целостность данных	Высокая (первичные, внешние ключи)	Зависит от реализации
Масштабируемость	Вертикальная	Горизонтальная
Пример использования	Финансовые системы, CRM, ERP	Большие данные, кеширование, социальные сети

Реляционная модель

- Сущность – например, клиенты, заказы, поставщики
- Таблица – отношение
- Столбец – атрибут
- Строка / запись – кортеж
- Результирующий набор – набор запроса SQL:


```
SELECT contact_name, address, city
FROM customers
LIMIT 13
```

SQL – Structured Query Language

- SQL – непроцедурный язык и не язык общего назначения
- Если необходимо реализовать процедурную логику – нужен другой язык, такой как Python, C#, Java и т.д.
- Результатом SQL запроса является результирующий набор (как правило – таблица)
- DDL (Data Definition Language) – CREATE, ALTER, DROP (категории SQL)
- DML (Data Manipulation Language) – SELECT, INSERT, UPDATE, DELETE
- TCL (Transaction Control Language) – COMMIT, ROLLBACK, SAVEPOINT (механизм, который дает целостность данных)
- DCL (Data Control Language) – GRANT, REVOKE, DENY
- ANSI SQL – 92 (общее подчасть всех SQL – подобных языков)
- Различия в процедурных расширениях: PL/pgSQL в PostgreSQL, PL/SQL в Oracle, T – SQL в MS SQL

Почему PostgreSQL?

- Free (Open Source)
- Лучший выбор для изучения: проинсталлировал и “понеслась”!
- “Взрослая” СУБД, хорошо поддерживающая транзакционность из коробки
- Весьма развитый диалект SQL
- В сравнении с MySQL есть свои плюсы и минусы
- В любом случае, 90% возможностей диалекта SQL поддерживаемого PostgreSQL можно без изменений использовать и в других СУБД

Типы данных в PostgreSQL

Тип данных	Группа	Размер	Описание	Диапазон значений
<code>smallint</code>	Числовой	2 bytes	Малое целое число	-32,768 до 32,767
<code>integer</code>	Числовой	4 bytes	Целое число	-2,147,483,648 до 2,147,483,647
<code>bigint</code>	Числовой	8 bytes	Большое целое число	-9,223,372,036,854,775,808 до 9,223,372,036,854,775,807
<code>decimal/numeric</code>	Числовой	Переменный	Десятичное число	Переменный
<code>real/float4</code>	Числовой	4 bytes	Число с плавающей запятой	~1.18E-38 до 3.4E+38
<code>double precision/float8/float</code>	Числовой	8 bytes	Число с двойной точностью	~2.23E-308 до 1.79E+308
<code>smallserial</code>	Числовой	2 bytes	Последовательность	1 до 32,767
<code>serial</code>	Числовой	4 bytes	Последовательность	1 до 2,147,483,647
<code>bigserial</code>	Числовой	8 bytes	Последовательность	1 до 9,223,372,036,854,775,807
<code>char</code>	Символьный	Переменный	Фиксированная строка	Переменный

Тип данных	Группа	Размер	Описание	Диапазон значений
text	Символьный	Переменный	Текстовое поле	Переменный
boolean/bool	Логический	1 byte	Логическое значение	True/False
date	Дата и время	4 bytes	Дата	4713 B.C. до 294,276 A.D.
time	Дата и время	8 bytes	Время	00:00:00 до 24:00:00
timestamp	Дата и время	8 bytes	Метка времени	4713 B.C. до 294,276 A.D.
interval	Дата и время	16 bytes	Интервал времени	-178,000.000 до +178,000.000
timestamptz	Дата и время	8 bytes	Метка времени с часовым поясом	4713 B.C. до 294,276 A.D.
Arrays	Специальные	Переменный	Массивы	Переменный
JSON	Специальные	Переменный	JSON	Переменный
XML	Специальные	Переменный	XML	Переменный
Геометрические типы и др. спец. типы	Специальные	Переменный	Разные	Переменный
Custom	Специальные	Переменный	Пользовательские типы	Переменный
NULL	Специальные	0 bytes	Отсутствие данных	-

Команды для работы с базами данных:

-
- \c [database_name]: Подключиться к базе данных.
 - \l или \list: Показать список баз данных.
 - \dn или \list-namespaces: Показать список схем в текущей базе данных.
 - \dt [schema_name.][table_name] или \list-tables [schema_name.][table_name]: Показать список таблиц.
 - \d [table_name]: Показать описание структуры таблицы.
 - \di [index_name] или \list-indexes [index_name]: Показать список индексов.
 - \dv [view_name] или \list-views [view_name]: Показать список представлений.

Команды для работы с данными:

-
- SELECT * FROM table_name;: Выполнить запрос на выборку данных из таблицы.
 - INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);: Вставить данные в таблицу.
 - UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;: Обновить данные в таблице.
 - DELETE FROM table_name WHERE condition;: Удалить данные из таблицы.

Команды для работы с транзакциями:

-
- BEGIN;: Начать транзакцию.
 - COMMIT;: Зафиксировать транзакцию.
 - ROLLBACK;: Откатить транзакцию.

Команды для работы с пользователями и ролями:

-
- \du или \list-roles: Показать список пользователей и ролей.
 - CREATE USER username WITH PASSWORD 'password';: Создать нового пользователя.
 - GRANT ALL PRIVILEGES ON database_name TO username;: Предоставить все привилегии пользователю на базе данных.

Системные команды:

-
- \q: Выйти из psql.
 - ! command: Выполнить системную команду.

Команды для настройки вывода:

- \pset [format]: Изменить формат вывода (например, \pset format aligned для табличного вывода).
- \timing: Включить или выключить отображение времени выполнения команд.

Полезные команды:

- ?: Показать справку по всем командам psql.
- \x [on|off]: Включить или выключить расширенный формат вывода.

Создание пользователя

Подключитмся к PostgreSQL

```
psql -h localhost -U postgres
```

Создать пользователя

```
CREATE USER myuser WITH PASSWORD 'mypassword';
```

Предоставить привилегии (опционально)

```
ALTER USER myuser CREATEDB;
```

```
ALTER USER myuser WITH SUPERUSER;
```

Назначение пользователя базе данных

```
GRANT ALL PRIVILEGES ON DATABASE mydatabase TO myuser;
```

При

```
CREATE USER myuser WITH PASSWORD 'mypassword';  
CREATE DATABASE mydatabase;  
GRANT ALL PRIVILEGES ON DATABASE mydatabase TO myuser;
```

Выйдите из psql

```
\q
```

Создание базы данных

```
CREATE DATABASE testdb  
WITH  
OWNER = root  
ENCODING = `UTF8`  
CONNECTION LIMIT = -1;
```

Удаление базы данных

```
DROP DATABASE testdb;
```

Создание таблиц

```
CREATE TABLE publisher (  
    publisher_id integer PRIMARY KEY,  
    org_name VARCHAR(128) NOT NULL,  
    address text NOT NULL);
```

```
CREATE table book (  
    book_id INTEGER PRIMARY KEY,  
    title VARCHAR(32) NOT NULL,  
    isbn text NOT NULL);
```

Удаление таблиц

```
DROP TABLE publisher;  
DROP TABLE book;
```

Создание записей

```
INSERT INTO publisher  
VALUES  
    (1, 'Everyman's Library', 'NY'),  
    (2, 'Oxford University Press', 'NY'),  
    (3, 'Grand Central Publishing', 'Washington'),  
    (4, 'Simon & Schuster', 'Chicago');
```

```
INSERT INTO book  
VALUES  
    (1, 'The diary of a Young Girl', '123456'),  
    (2, 'Pride and Prejudice', '234353535'),  
    (3, 'To Kill a Mockingbird', '23242424'),  
    (4, 'The Book of Gutsy Women', '232424242'),  
    (5, 'War and Peace', '131313143');
```

Отношения

1. Отношение «Один ко многим» (One-to-Many)

```
CREATE TABLE Authors (  
    AuthorID SERIAL PRIMARY KEY,  
    Name VARCHAR(100)  
);  
  
CREATE TABLE Books (  
    BookID SERIAL PRIMARY KEY,  
    Title VARCHAR(100),  
    AuthorID INT REFERENCES Authors(AuthorID)  
);
```

2. Отношение «Один к одному» (One-to-One)

```
CREATE TABLE Users (  
    UserID SERIAL PRIMARY KEY,  
    Username VARCHAR(100)  
);  
  
CREATE TABLE UserProfiles (  
    ProfileID SERIAL PRIMARY KEY,  
    UserID INT UNIQUE REFERENCES Users(UserID),  
    Bio TEXT  
);
```

В данном случае связь "один к одному" обеспечивается тем, что внешний ключ (UserID) уникален в таблице UserProfiles. Это гарантирует, что каждая запись в Users соответствует ровно одной записи в UserProfiles.

3. Отношение «Многие ко многим» (Many-to-Many)

```
CREATE TABLE Students (  
    StudentID SERIAL PRIMARY KEY,  
    Name VARCHAR(100)  
);  
  
CREATE TABLE Courses (  
    CourseID SERIAL PRIMARY KEY,  
    Title VARCHAR(100)  
);  
  
CREATE TABLE StudentCourses (  
    StudentID INT REFERENCES Students(StudentID),  
    CourseID INT REFERENCES Courses(CourseID),  
    PRIMARY KEY (StudentID, CourseID)  
);
```

В данной реализации, связь "многие ко многим" осуществляется через промежуточную таблицу StudentCourses, где каждый студент может быть записан на несколько курсов, и каждый курс может включать несколько студентов.

Ключевое слово DISTINCT

В PostgreSQL, ключевое слово DISTINCT используется в SQL-запросах для удаления дублирующихся строк из результатов запроса. Если в наборе данных есть повторяющиеся строки, DISTINCT оставит только одну из них, удаляя остальные.

```
SELECT DISTINCT column1, column2  
FROM table_name;
```

Pattern Matching with LIKE(чувствителен к регистру) and ILIKE (не чувствителен к регистру)

- % - placeholder (заполнитель) означающий 0, 1 и более символов
- _ - ровно 1 любой символ
- LIKE 'U%' - строки, начинающиеся с U
- LIKE 'a%' - строки, начинающиеся с a
- LIKE '%John%' - строки, содержащие John
- LIKE 'J%n' - строки, начинающиеся с J, и оканчивающиеся на n
- LIKE '_oh_' - строки, где 2, 3 символы - oh, а первый(1) и последний(4) - любые
- LIKE '_oh%' - строки, где 2, 3 символы - oh, первый - любой и в конце 0, 1 и более любых символов